Collection Framework Architecture
Java Lecture-2

Agenda

- Enhanced For Loop
- Collection Framework
  - Internal Architecture
  - Iterator Interface and
  - Iterable Interface
- Quiz
- Homework

Enhanced For Loop

- In Java, there is another form of for loop (in addition to standard for loop) to work with arrays and collection, the **enhanced for** loop.
- **Enhanced for** loop is also known as for-each loop and was introduced in Java 5.
- **Syntax of enhanced for loop:**

```
for(<collection_type> <tempvar for each object from index 0> : <collection_name>)
{
  // statements
}
```

- **Enhanced for** loop always starts from index **0** and iterates over the complete collection indexes.
- This loop is used for **non-primitive** collections as we have to give class type, but we also use it for **primitive arrays**.

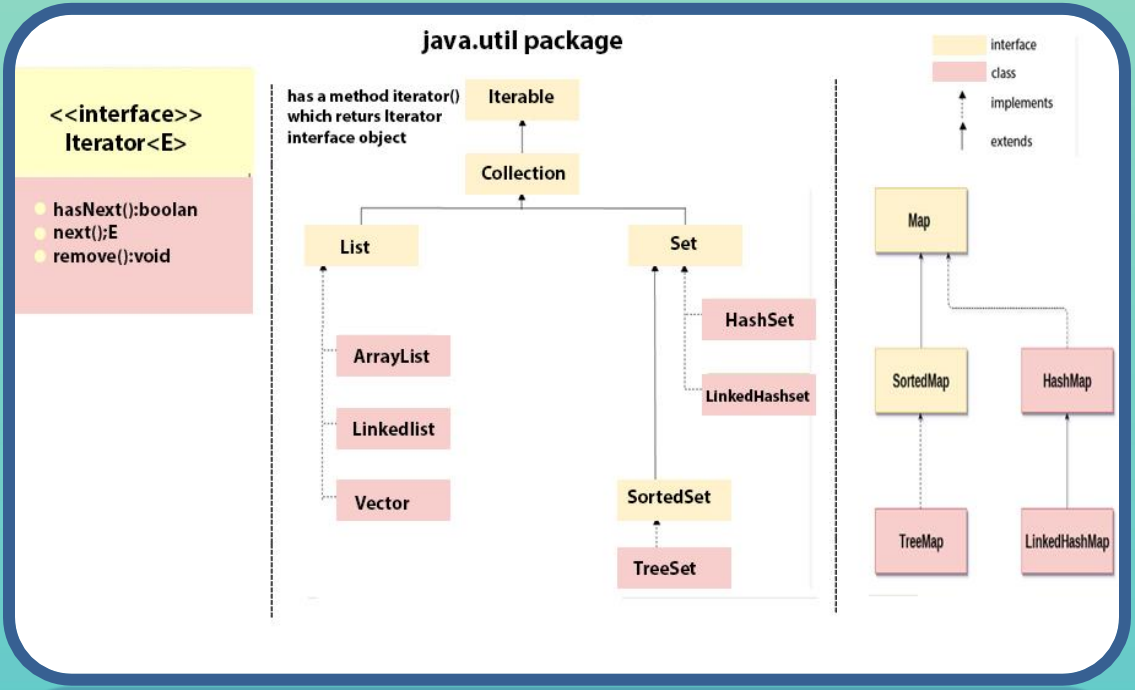
```
String[] schoolbagColl = { "Books", "Pens", "Pencils", "Notebooks" };
// iteration provided by Java 5 or later
for (String obj : schoolbagColl) {    //using enhanced for loop
    System.out.println(obj+" "+"pp");
}
```

Collection Framework

- The **Collection in Java** is a framework that provides an architecture to store and manipulate the **group of objects as a single unit**
- **Collection Framework** is a collection of **interfaces and classes** which helps in storing, searching, sorting, inserting, deleting and processing the data efficiently
- **Collection classes and interfaces** have tons of useful methods which can be called and reused whenever required
- **Collection** are set of **classes & interfaces** that provide readymade architecture and is optional to use
- In **Collection Framework,** all **classes and interfaces** are present in **java.util** package and they stores only class objects and not primitives
- Three most important **interfaces** in **Collection Framework** are:
  - **List interface**
  - **Set interface**
  - **Map interface**

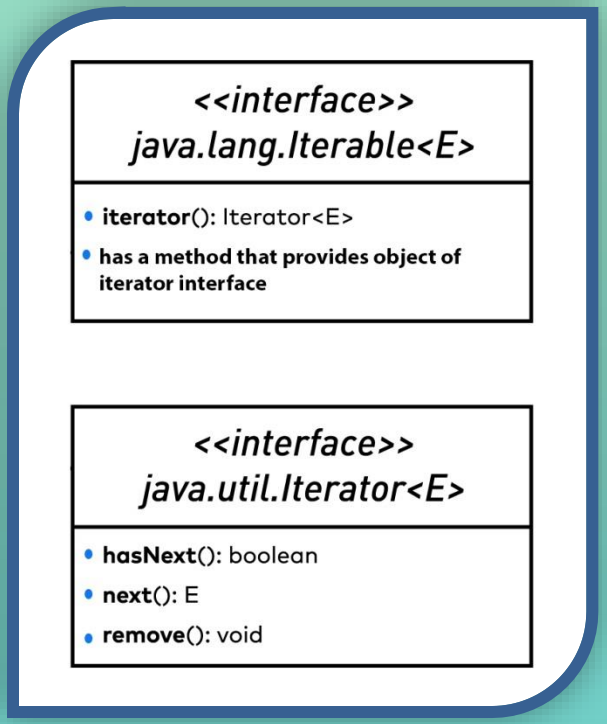
Collection Framework Architecture

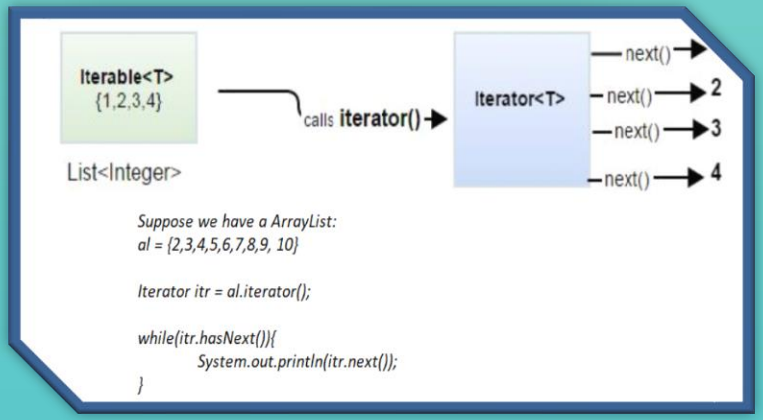
- **Collection interface** is parent of:
  - **List interface**
  - **Set interface**
- **List Interface** is implemented by:
  - **ArrayList class**
  - **LinkedList class**
  - **Vector class**
- **Vector class** is **synchronised**, therefore it is not frequently used
- **ArrayList** and **LinkedList class** are most used implementation of List interface
- **Map interface** is separately placed in **java.util** package and not in hierarchy of **Collection interface**



Iterator Interface and Iterable Interface

- **Iterable Interface** – It is at the top level of hierarchy of **Collection Interface**
- All the implementations of **Collection (List & Set)** interface also implements **Iterable interface**
- It has an abstract method **iterator()** that returns object of **Iterator interface**
- Ability to read **Java Collection** is possible only due to implementation of **Iterable interface**
- 2 Ways to iterate Java Collection are through:
  - **Enhanced for loop**
  - **Iterator interface object**



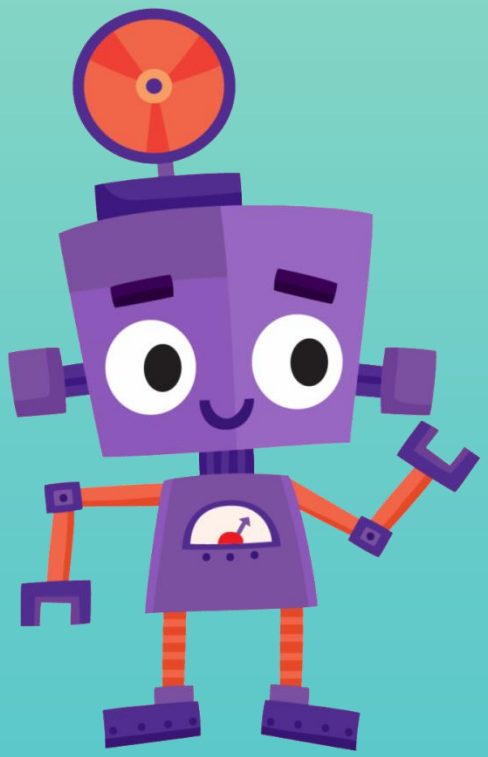


Quiz

- Can you explain the functioning of **Enhanced for** loop?
- Can **Enhanced for** loop be used for **primitive** type of collection?
- What are the three most important **interfaces in Collection Framework**?
- Which interfaces are part of **Collection interface** hierarchy?
- Can you please tell me the classes implemented by **List interface**?
- What is the importance of **Iterable interface**?
- How can the **Collection interface** object be read with the help of **Iterator interface**?
- Which two implementations of **Collection interface** can use **Iterator interface**?

Homework

- Create class “**Book**” with its **default constructor** initializing 3 instance variables – “**title(String)**”, “**price(int)**”, “**author(String)**”. Create class “**BookTest**” with the main method and:
  - Create **5** objects of “**Book**” class and put them in array.
  - Read the **array** with **enhanced for loop** and print on console “**Book**” object all attributes.

- Go through **Collection Framework** hierarchy and list the importance of **Iterable interface.**

- Please write down the **methods** and their **return type** of **Iterator interface.**